MyCodeSchool

Exerciții — backtracking: permutări

Recapitulare · C++ · backtracking — permutări · 5 algoritmi, 25 probleme

`algoritmi.h` e doar de studiu. Are înăuntru schema comună și câte un algoritm de bază pe secțiune, gata scriși, cu urme de trasare a lui `x[]`. Îl citești, îl închizi, apoi **scrii tu** algoritmul în headerul tău. Dacă te blochezi, îl trasezi pe hârtie, nu-l copiezi.

Forma de bac: vectorul soluție `x[]` e global, indexat de la 1; fiecare problemă are `valid(k)` și `back(k)`, iar datele ei sunt în variabilele globale de mai jos, pe care le pune `main` înainte de apel. Primul fișier, `solutie.h`, îl scrii o dată; toate headerele îl includ:

```
#ifndef SOLUTIE_H_INCLUDED
#define SOLUTIE_H_INCLUDED

#include <iostream>

using namespace std;

int n, m, s, t, a[20], x[20];
char l[20];

bool nou(int k)
{
    for (int i = 1; i < k; i++)
    {
        if (x[i] == x[k])
        {
            return false;
        }
    }

    return true;
}

void afisare(int lungime)
{
    for (int i = 1; i <= lungime; i++)
    {
        if (i > 1)
        {
            cout << " ";
        }
        cout << x[i];
    }
    cout << endl;
}

#endif // SOLUTIE_H_INCLUDED
```

Cum lucrezi la fișă:

- **pentru fiecare secțiune îți faci headerul ei** — numele e scris în titlul secțiunii;
- sus în header scrii **tu** algoritmul de bază din `algoritmi.h`, apoi cele 5 probleme;
- fiecare problemă = funcția cerută (fără parametri, ca în fișă) + `valid` și `back` ale ei, cu numele problemei în coadă (`validPrimImpar`, `backPrimImpar`); altfel, incluse împreună, dau `redefinition`;
- funcția cerută **afișează** soluțiile, câte una pe linie, în ordine lexicografică, cu elementele separate prin spațiu;
- în `main.cpp` incluzi headerul la care lucrezi, pui valorile din exemplu în `n`, `m` … și chemi funcția;

- garda se scrie fără cratimă (`PERMUTARI_CLASIC_H_INCLUDED`), ca la celelalte lecții;
- build cu **Ctrl+Shift+B** (pe Mac **Cmd+Shift+B**; task-ul `build` din `.vscode/tasks.json`), rulezi din terminal cu `./main` .

Reguli:

- **Nu ai voie** să incluzi `algoritmi.h` , să chemi ceva din el sau să copiezi din el.
- **Ai voie** să chemi funcțiile **tale**, din headerele tale, oricâte, în orice combinație.
- **Nu ai voie:** `next_permutation` , `#include <algorithm>` , `vector` , `string` .
- La ieșire, unde sunt multe soluții (peste 12), fișa arată primele 5 și numărul total; restul le verifici după **numărul** lor.
- Limită: `n` ≤ 19 (vectorul `x[20]`).

1. Permutările clasice · `permutari_clasic.h`

Studiezi din `algoritmi.h` : `permutari`

Recunoști: **permutări**, `valid(k)` = `nou(k)` și condiția problemei.

Primul impar, ultimul par

1.1

Afișează, în ordine lexicografică, permutările mulțimii {1, 2, …, `n` } în care primul element este impar, iar ultimul este par.

`void permutariImparPar()`

```
n = 4  ->  1 2 3 4
           1 3 2 4
           1 3 4 2
           1 4 3 2
           3 1 2 4
           3 1 4 2
           3 2 1 4
           3 4 1 2
n = 3  ->  1 3 2
           3 1 2
n = 2  ->  1 2
n = 1  ->  (nimic)
```

Primul mai mic decât ultimul

1.2

Afișează, în ordine lexicografică, permutările mulțimii {1, 2, …, `n`} în care primul element este strict mai mic decât ultimul.

`void permutariPrimMic()`

```
n = 3  ->  1 2 3
           1 3 2
           2 1 3
n = 4  ->  1 2 3 4
           1 2 4 3
           1 3 2 4
           1 3 4 2
           1 4 2 3
           1 4 3 2
           2 1 3 4
           2 1 4 3
           2 3 1 4
           2 4 1 3
           3 1 2 4
           3 2 1 4
n = 2  ->  1 2
n = 1  ->  (nimic)
```

Unu înaintea lui n

1.3

Afișează, în ordine lexicografică, permutările mulțimii {1, 2, …, `n`} (`n` ≥ 2) în care elementul 1 apare înaintea elementului `n`.

`void permutariUnuInainte()`

```
n = 2  ->  1 2
n = 3  ->  1 2 3
           1 3 2
           2 1 3
n = 4  ->  1 2 3 4
           1 2 4 3
           1 3 2 4
           1 3 4 2
           1 4 2 3
           1 4 3 2
           2 1 3 4
           2 1 4 3
           2 3 1 4
           3 1 2 4
           3 1 4 2
           3 2 1 4
n = 5  ->  1 2 3 4 5
           1 2 3 5 4
           1 2 4 3 5
           1 2 4 5 3
           1 2 5 3 4
           ... (60 soluții)
```

Fără punct fix

1.4

Afișează, în ordine lexicografică, permutările mulțimii {1, 2, …, `n`} în care niciun element nu stă pe poziția egală cu el (`x[k] != k`).

`void permutariFaraPunctFix()`

```
n = 3  ->  2 3 1
           3 1 2
n = 4  ->  2 1 4 3
           2 3 4 1
           2 4 1 3
           3 1 4 2
           3 4 1 2
           3 4 2 1
           4 1 2 3
           4 3 1 2
           4 3 2 1
n = 2  ->  2 1
n = 1  ->  (nimic)
```

Exact t puncte fixe

1.5

Afișează, în ordine lexicografică, permutările mulțimii {1, 2, …, `n`} în care exact `t` elemente stau pe poziția egală cu ele (`x[k] == k`).

`void permutariPuncteFixe()`

```
n = 4 , t = 2  ->  1 2 4 3
                   1 3 2 4
                   1 4 3 2
                   2 1 3 4
                   3 2 1 4
                   4 2 3 1
n = 3 , t = 1  ->  1 3 2
                   2 1 3
                   3 2 1
n = 3 , t = 2  ->  (nimic)
n = 4 , t = 4  ->  1 2 3 4
```

2. Condiții pe poziții · `permutari-pozitii.h`

Studiezi din `algoritmi.h` : `permutariPrimEgal`

Recunoști: **permutări**, condiția stă pe `x[k]` raportat la `k` sau la ce a fost deja pus.

Elementul t pe poziția m

2.1

Afișează, în ordine lexicografică, permutările mulțimii {1, 2, …, `n`} (1 ≤ `m` ≤ `n`) în care pe poziția `m` se află elementul `t`.

`void permutariPePozitie()`

```
n = 4 , m = 2 , t = 3  ->  1 3 2 4
                           1 3 4 2
                           2 3 1 4
                           2 3 4 1
                           4 3 1 2
                           4 3 2 1
n = 3 , m = 3 , t = 1  ->  2 3 1
                           3 2 1
n = 3 , m = 1 , t = 2  ->  2 1 3
                           2 3 1
n = 1 , m = 1 , t = 1  ->  1
```

Paritatea poziției

2.2

Afișează, în ordine lexicografică, permutările mulțimii {1, 2, …, `n`} în care fiecare element are aceeași paritate cu poziția lui (`x[k] % 2 == k % 2`).

`void permutariParitate()`

```
n = 3  ->  1 2 3
           3 2 1
n = 4  ->  1 2 3 4
           1 4 3 2
           3 2 1 4
           3 4 1 2
n = 5  ->  1 2 3 4 5
           1 2 5 4 3
           1 4 3 2 5
           1 4 5 2 3
           3 2 1 4 5
           3 2 5 4 1
           3 4 1 2 5
           3 4 5 2 1
           5 2 1 4 3
           5 2 3 4 1
           5 4 1 2 3
           5 4 3 2 1
n = 1  ->  1
```

Cel mult t poziții de locul lui

2.3

Afișează, în ordine lexicografică, permutările mulțimii {1, 2, …, `n`} în care fiecare element e la cel mult `t` poziții de locul lui (`|x[k] - k| ≤ t`).

`void permutariDeplasare()`

```
n = 4 , t = 1  ->  1 2 3 4
                   1 2 4 3
                   1 3 2 4
                   2 1 3 4
                   2 1 4 3
n = 3 , t = 1  ->  1 2 3
                   1 3 2
                   2 1 3
n = 4 , t = 0  ->  1 2 3 4
n = 4 , t = 2  ->  1 2 3 4
                   1 2 4 3
                   1 3 2 4
                   1 3 4 2
                   1 4 2 3
                   ... (14 soluții)
```

Pare în ordine crescătoare

2.4

Afișează, în ordine lexicografică, permutările mulțimii {1, 2, …, `n`} în care numerele pare apar în ordine crescătoare, de la stânga la dreapta (cele impare pot fi oriunde).

`void permutariPareCrescatoare()`

```
n = 3  ->  1 2 3
           1 3 2
           2 1 3
           2 3 1
           3 1 2
           3 2 1
n = 4  ->  1 2 3 4
           1 2 4 3
           1 3 2 4
           2 1 3 4
           2 1 4 3
           2 3 1 4
           2 3 4 1
           2 4 1 3
           2 4 3 1
           3 1 2 4
           3 2 1 4
           3 2 4 1
n = 5  ->  1 2 3 4 5
           1 2 3 5 4
           1 2 4 3 5
           1 2 4 5 3
           1 2 5 3 4
           ... (60 soluții)
n = 1  ->  1
```

A înaintea lui B

2.5

Afișează, în ordine lexicografică, permutările mulțimii {1, 2, …, `n`} în care, pentru fiecare dintre cele `m` perechi (`a[2i-1]`, `a[2i]`), elementul `a[2i-1]` apare înaintea lui `a[2i]`. Perechile se dau în `a[1..2m]` (`2m` ≤ 19).

`void permutariPrecedente()`

```
n = 4 , a = 1 3 2 4 , m = 2  ->  1 2 3 4
                                 1 2 4 3
                                 1 3 2 4
                                 2 1 3 4
                                 2 1 4 3
                                 2 4 1 3
n = 3 , a = 1 2 2 3 3 1 , m = 3  ->  (nimic)
n = 3 , a = 3 1 , m = 1  ->  2 3 1
                             3 1 2
                             3 2 1
n = 4 , a = 1 2 2 3 3 4 , m = 3  ->  1 2 3 4
```

3. Condiții pe vecini · `permutari-vecini.h`

Studiezi din `algoritmi.h`: `permutariVecini`

Recunoști: **permutări**, `valid(k)` compară `x[k]` cu `x[k-1]` (doar când `k > 1`).

1 și n alăturate

3.1

Afișează, în ordine lexicografică, permutările mulțimii {1, 2, …, `n`} (`n` ≥ 2) în care elementele 1 și `n` sunt vecine, în oricare ordine.

`void permutariUnuSiN()`

```
n = 3  ->  1 3 2
           2 1 3
           2 3 1
           3 1 2
n = 4  ->  1 4 2 3
           1 4 3 2
           2 1 4 3
           2 3 1 4
           2 3 4 1
           2 4 1 3
           3 1 4 2
           3 2 1 4
           3 2 4 1
           3 4 1 2
           4 1 2 3
           4 1 3 2
n = 2  ->  1 2
           2 1
n = 5  ->  1 5 2 3 4
           1 5 2 4 3
           1 5 3 2 4
           1 5 3 4 2
           1 5 4 2 3
           ... (48 soluții)
```

Fără trei în creștere

3.2

Afișează, în ordine lexicografică, permutările mulțimii {1, 2, …, `n`} în care nu există trei elemente vecine în ordine strict crescătoare.

`void permutariFaraTreiCrescatoare()`

```
n = 3  ->  1 3 2
           2 1 3
           2 3 1
           3 1 2
           3 2 1
n = 4  ->  1 3 2 4
           1 4 2 3
           1 4 3 2
           2 1 4 3
           2 3 1 4
           ... (17 soluții)
n = 2  ->  1 2
           2 1
n = 1  ->  1
```

Permutări în zigzag

3.3

Afișează, în ordine lexicografică, permutările mulțimii {1, 2, …, `n`} în care `x[1] < x[2] > x[3] < x[4] > …`

`void permutariZigzag()`

```
n = 3  ->  1 3 2
           2 3 1
n = 4  ->  1 3 2 4
           1 4 2 3
           2 3 1 4
           2 4 1 3
           3 4 1 2
n = 5  ->  1 3 2 5 4
           1 4 2 5 3
           1 4 3 5 2
           1 5 2 4 3
           1 5 3 4 2
           ... (16 soluții)
n = 1  ->  1
```

Vecini la distanță cel puțin t

3.4

Afișează, în ordine lexicografică, permutările mulțimii {1, 2, …, `n`} în care diferența în modul dintre oricare doi vecini este cel puțin `t`.

`void permutariVeciniDistanta()`

```
n = 4 , t = 2  ->  2 4 1 3
                   3 1 4 2
n = 4 , t = 3  ->  (nimic)
n = 5 , t = 3  ->  (nimic)
n = 3 , t = 1  ->  1 2 3
                   1 3 2
                   2 1 3
                   2 3 1
                   3 1 2
                   3 2 1
```

Cerc cu parități alternate

3.5

Afișează, în ordine lexicografică, permutările mulțimii {1, 2, …, `n`} (`n` ≥ 2) așezate în cerc (ultimul e vecin cu primul), în care oricare doi vecini au parități diferite.

`void permutariCercParitati()`

```
n = 4  ->  1 2 3 4
           1 4 3 2
           2 1 4 3
           2 3 4 1
           3 2 1 4
           3 4 1 2
           4 1 2 3
           4 3 2 1
n = 3  ->  (nimic)
n = 2  ->  1 2
           2 1
n = 6  ->  1 2 3 4 5 6
           1 2 3 6 5 4
           1 2 5 4 3 6
           1 2 5 6 3 4
           1 4 3 2 5 6
           ... (72 soluții)
```

4. Permutări ale unei mulțimi date · `permutari-multime.h`

Studiezi din `algoritmi.h`: `permutariVector, afisareValori`

Recunoști: **permutări de indici**: `x[k]` e un indice, valoarea e `a[x[k]]`.

Primul și ultimul au suma s

4.1

Afișează permutările vectorului `a[1..n]` (elemente distincte), în ordinea indicilor, scriind valorile în care primul și ultimul element au suma `s`.

`void permutariSumaExtreme()`

```
a = 1 2 3 4 , s = 5  ->  1 2 3 4
                         1 3 2 4
                         2 1 4 3
                         2 4 1 3
                         3 1 4 2
                         3 4 1 2
                         4 2 3 1
                         4 3 2 1
a = 5 8 1 , s = 6  ->  5 8 1
                       1 8 5
a = 5 8 1 , s = 13  ->  5 1 8
                        8 1 5
a = 4 7 2 , s = 100  ->  (nimic)
```

Pare și impare alternate

4.2

Afișează permutările vectorului `a[1..n]` (elemente distincte), în ordinea indicilor, scriind valorile în care elementele vecine au parități diferite.

`void permutariAlternante()`

```
a = 1 2 3 4  ->  1 2 3 4
                 1 4 3 2
                 2 1 4 3
                 2 3 4 1
                 3 2 1 4
                 3 4 1 2
                 4 1 2 3
                 4 3 2 1
a = 2 4 1  ->  2 1 4
               4 1 2
a = 1 3 5  ->  (nimic)
a = 6  ->  6
```

Maximul nu e la margine

4.3

Afișează permutările vectorului `a[1..n]` (elemente distincte), în ordinea indicilor, scriind valorile în care cel mai mare element nu este nici primul, nici ultimul.

`void permutariMaximInterior()`

```
a = 5 9 2  ->  5 9 2
               2 9 5
a = 4 1 7 3  ->  4 1 7 3
                 4 7 1 3
                 4 7 3 1
                 4 3 7 1
                 1 4 7 3
                 1 7 4 3
                 1 7 3 4
                 1 3 7 4
                 3 4 7 1
                 3 1 7 4
                 3 7 4 1
                 3 7 1 4
a = 6 2  ->  (nimic)
a = 8  ->  (nimic)
```

Suma vecinilor este număr prim

4.4

Afișează permutările vectorului `a[1..n]` (elemente distincte), în ordinea indicilor, scriind valorile în care suma oricăror două elemente vecine este număr prim.

`void permutariSumaPrima()`

```
a = 1 2 3 4  ->  1 2 3 4
                 1 4 3 2
                 2 1 4 3
                 2 3 4 1
                 3 2 1 4
                 3 4 1 2
                 4 1 2 3
                 4 3 2 1
a = 2 4 6  ->  (nimic)
a = 1 2 3  ->  1 2 3
               3 2 1
a = 5  ->  5
```

Primele m în ordine crescătoare

4.5

Afișează permutările vectorului `a[1..n]` (elemente distincte), în ordinea indicilor, scriind valorile în care primele `m` elemente sunt în ordine strict crescătoare (`m` ≤ `n`).

`void permutariPrimeleCrescatoare()`

```
a = 4 1 3 , m = 2  ->  1 4 3
                       1 3 4
                       3 4 1
a = 5 2 8 1 , m = 3  ->  2 5 8 1
                         1 5 8 2
                         1 2 5 8
                         1 2 8 5
a = 7 3 , m = 1  ->  7 3
                     3 7
a = 6 2 9 , m = 3  ->  2 6 9
```

5. Anagrame · `permutari-anagrame.h`

Studiezi din `algoritmi.h`: `anagrame, afisareLitere, anagrameDistincte`

Recunoști: **permutări de litere**; la litere care se repetă, un indice nu se pune înaintea celui identic cu el.

Anagrame care încep cu vocală — 5.1

Afișează anagramele cuvântului `l[1..n]` (litere distincte), în ordinea indicilor, fără spații, doar pe cele care încep cu o vocală (a, e, i, o, u).

`void anagrameVocalaInceput()`

```
l = b a l e  ->  able
                 abel
                 albe
                 aleb
                 aebl
                 aelb
                 ebal
                 ebla
                 eabl
                 ealb
                 elba
                 elab
l = r a c  ->  arc
               acr
l = x y z  ->  (nimic)
l = a e  ->  ae
             ea
```

Fără două vocale alăturate — 5.2

Afișează anagramele cuvântului `l[1..n]` (litere distincte), în ordinea indicilor, fără spații, doar pe cele fără două vocale vecine.

`void anagrameFaraVocaleAlaturate()`

```
l = m a r e  ->  mare
                 mera
                 amre
                 amer
                 arme
                 arem
                 rame
                 rema
                 emar
                 emra
                 erma
                 eram
l = a b c  ->  abc
               acb
               bac
               bca
               cab
               cba
l = a e i  ->  (nimic)
l = a b  ->  ab
             ba
```

Vocalele în ordine alfabetică

5.3

Afișează anagramele cuvântului `l[1..n]` (litere distincte), în ordinea indicilor, fără spații, doar pe cele în care vocalele apar în ordine alfabetică (consoanele pot fi oriunde).

`void anagrameVocaleAlfabetic()`

```
l = m a r e  ->  mare
                 maer
                 mrae
                 amre
                 amer
                 arme
                 arem
                 aemr
                 aerm
                 rmae
                 rame
                 raem
l = i o a  ->  aio
l = c d f  ->  cdf
               cfd
               dcf
               dfc
               fcd
               fdc
l = e a  ->  ae
```

Anagrame distincte

5.4

Afișează, în ordine alfabetică, anagramele distincte ale cuvântului `l[1..n]`, ale cărui litere sunt în ordine alfabetică și se pot repeta.

`void anagrameDistincte()`

```
l = a a b  ->  aab
               aba
               baa
l = a a b b  ->  aabb
                 abab
                 abba
                 baab
                 baba
                 bbaa
l = a b c  ->  abc
               acb
               bac
               bca
               cab
               cba
l = a a a  ->  aaa
```

Anagrame palindrom

5.5

Afișează, în ordine alfabetică, anagramele distincte ale cuvântului `l[1..n]` (litere în ordine alfabetică, care se pot repeta) care sunt palindroame.

`void anagramePalindrom()`

```
l = a a b b c  ->  abcba
                   bacab
l = a a b b  ->  abba
                 baab
l = a b c  ->  (nimic)
l = a a b  ->  aba
```

Pentru fiecare problemă pe care o scrii, verific-o pe:

1. cel mai mic caz (**n = 1** sau **m = 1**);
2. un caz fără nicio soluție: funcția **nu afișează nimic**, fără să se blocheze;
3. **numărul** soluțiilor (îl numeri cu un contor în `main` sau cu `| wc -l`) și **ordinea** lor, comparate cu ce ai calculat pe hârtie;
4. un `valid(k)` care compară cu `x[k-1]` doar când **k > 1**.